

Jira Task 05 – Build Basic Java Microservices System

> Implementation

Step 1: Create Spring boot Projects

For this microservices project, create **two separate Spring Boot projects**:

- user-service
- order-service

Project 1: user-service

Go to **the Spring Initializer** in your browser.

Select:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** 3.x
- **Group:** com.example
- **Artifact:** user-service
- **Name:** user-service
- **Packaging:** Jar
- **Java:** 17

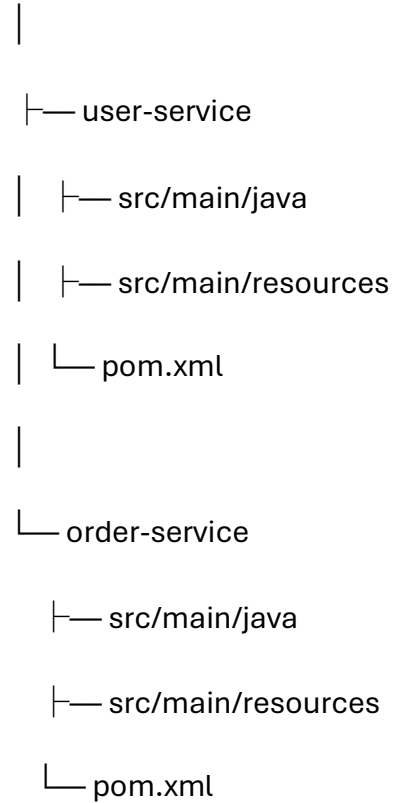
Add dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver

Generate, extract, and import it into Eclipse.

Your Eclipse should look approximately like:

Eclipse



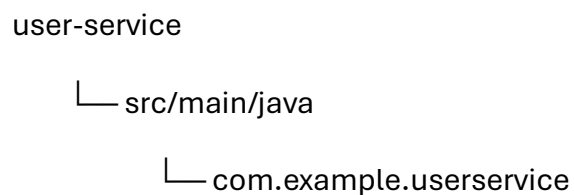
Step 1 is complete when both projects are created and imported into Eclipse.

Step 2: Create User Service

Now we will create the **User Service** inside your user-service Spring Boot project.

1. Open user-service in Eclipse

In **Project Explorer**, open:



2. Create the User package

Right-click your main package:

New → Package

Enter:

com.example.userservice.controller

Click **Finish**.

Then create another package:

com.example.userservice.service

3. Create User Service class

Right-click the service package:

New → Class

Name:

UserService

Click **Finish**.

Put this code:

```
package com.example.userservice.service;
```

```
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class UserService {
```

```
public String getUser() {  
    return "User Service is working";  
}  
}
```

4. Create UserController

Right-click:

com.example.userservice.controller

Choose:

New → Class

Name:

UserController

Use:

```
package com.example.userservice.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
import com.example.userservice.service.UserService;
```

```
@RestController
```

```
public class UserController {
```

```
private final UserService userService;

public UserController(UserService userService) {

    this.userService = userService;

}

@GetMapping("/users")

public String getUser() {

    return userService.getUser();

}

}
```

5. Run the application

Find your main application class, something like:

UserServiceApplication.java

Right-click it → **Run As → Spring Boot App.**

Then open your browser and go to:

<http://localhost:8080/users>

You should see:

User Service is working

Step 3 — Create Order Service

Now we will create the **Order Service** in the order-service project.

1. Open order-service in Eclipse

In **Project Explorer**, expand:

order-service

└─ src/main/java

└─ com.example.orderservice

2. Create the Controller package

Right-click `com.example.orderservice` → **New** → **Package**

Enter:

`com.example.orderservice.controller`

Click **Finish**.

3. Create the Service package

Right-click `com.example.orderservice` → **New** → **Package**

Enter:

`com.example.orderservice.service`

Click **Finish**.

4. Create OrderService

Right-click the service package → **New** → **Class**

Name it:

`OrderService`

Put this code:

```
package com.example.orderservice.service;  
  
import org.springframework.stereotype.Service;
```

```
@Service
```

```
public class OrderService {  
  
    public String getOrder() {  
        return "Order Service is working";  
    }  
}
```

5. Create OrderController

Right-click the controller package → **New → Class**

Name it:

OrderController

Put this code:

```
package com.example.orderservice.controller;  
  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
import com.example.orderservice.service.OrderService;
```

```
@RestController
```

```
public class OrderController {
```

```
    private final OrderService orderService;
```

```
    public OrderController(OrderService orderService) {
```

```
        this.orderService = orderService;
```

```
    }
```

```
    @GetMapping("/orders")
```

```
    public String getOrder() {
```

```
        return orderService.getOrder();
```

```
    }
```

```
}
```

6. Run Order Service

Find:

OrderServiceApplication.java

Right-click → **Run As → Spring Boot App**

Then open:

<http://localhost:8080/orders>

You should get:

Order Service is working

If your user-service is already running port 8080, don't run both services on the same port. We can change the Order Service to **8081** before testing.

Step 4 — Configure Databases

Now we will configure a **MySQL database** for your two Spring Boot services.

We will use:

- userdb → for **User Service**
- orderdb → for **Order Service**

1. Create the databases in MySQL

Open **MySQL Workbench** and run:

```
CREATE DATABASE userdb;
```

```
CREATE DATABASE orderdb;
```

You should see:

userdb

orderdb

2. Configure user-service

In Eclipse, open:

user-service

→ src/main/resources

→ application.properties

Add:

spring.datasource.url=jdbc:mysql://localhost:3306/userdb

spring.datasource.username=root

spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

Replace:

YOUR_MYSQL_PASSWORD

With the password you created for MySQL.

3. Configure order-service

Open:

order-service

→ src/main/resources

→ application.properties

Add: spring.datasource.url=jdbc:mysql://localhost:3306/orderdb

spring.datasource.username=root

spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Again, replace YOUR_MYSQL_PASSWORD with your MySQL password.

4. Avoid port conflict

Because both services are Spring Boot applications, give them different ports.

For **user-service**:

```
server.port=8080
```

For **order-service**:

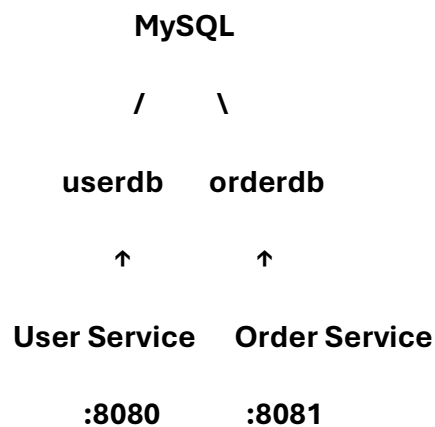
```
server.port=8081
```

So, your services will be:

User Service → http://localhost:8080

Order Service → <http://localhost:8081>

Your setup should now be:



Step 5 — Create Entities

Now we will create database **entities** for both microservices.

We need:

- User entity → user-service
- Order entity → order-service

Part 1 — User Entity

In Eclipse:

user-service → **src/main/java** → **com.example.userservice**

Right-click the package → **New** → **Package**

Create:

com.example.userservice.entity

Then right-click the entity package → **New** → **Class**

Name:

User

Replace the class code with:

```
package com.example.userservice.entity;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

@Entity

```
public class User {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

```
private String name;
```

```
private String email;
```

```
public User() {
```

```
}
```

```
public User(String name, String email) {
```

```
    this.name = name;
```

```
    this.email = email;
```

```
}
```

```
public Long getId() {
```

```
    return id;
```

```
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public void setEmail(String email) {  
    this.email = email;  
}  
}
```

Part 2 — Order Entity

In Eclipse:

order-service → **src/main/java** → **com.example.orderservice**

Create a package:

com.example.orderservice.entity

Then create a class:

Order

Put this code:

```
package com.example.orderservice.entity;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

```
@Entity
```

```
public class Order {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private Long userId;
```

```
    private String product;
```

```
    private Double amount;
```

```
    public Order() {
```

```
}
```

```
public Order(Long userId, String product, Double amount) {
```

```
    this.userId = userId;
```

```
    this.product = product;
```

```
    this.amount = amount;
```

```
}
```

```
public Long getId() {
```

```
    return id;
```

```
}
```

```
public Long getUserId() {
```

```
    return userId;
```

```
}
```

```
public void setUserId(Long userId) {
```

```
    this.userId = userId;
```

```
}
```

```
public String getProduct() {
```

```
    return product;
```



```

    }

    public void setProduct(String product) {

        this.product = product;

    }


    public Double getAmount() {

        return amount;

    }


    public void setAmount(Double amount) {

        this.amount = amount;

    }

}

```

Your Eclipse structure should now be

user-service

```

└─ src/main/java
    └─ com.example.userservice
        └─ entity
            └─ User.java

```

order-service

```

└─ src/main/java
    └─ com.example.orderservice

```

└─ entity
 └─ Order.java

When the applications run, JPA/Hibernate will create/update the corresponding tables in:

userdb → User table

orderdb → Order table

provided your `application.properties` from **Step 4** are configured correctly.

Step 6 — Create Repositories

Now we create the **Repository layer** for both services.

The repository connects our entities (User and Order) to the MySQL database.

1. Create User Repository

In Eclipse, go to:

user-service

→ `src/main/java`

→ `com.example.userservice`

Right-click the package → **New → Package**

Enter:

`com.example.userservice.repository`

Click **Finish**.

Now right-click the repository package:

New → Interface

Name it:

UserRepository

Put this code:

```
package com.example.userservice.repository;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.example.userservice.entity.User;
```

```
public interface UserRepository extends JpaRepository<User, Long> {
```

```
}
```

Save with **Ctrl + S**.

2. Create Order Repository

Go to:

order-service

→ src/main/java

→ com.example.orderservice

Create a package:

com.example.orderservice.repository

Then:

Right-click repository → New → Interface

Name:

OrderRepository

Put this code:

```
package com.example.orderservice.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.orderservice.entity.Order;

public interface OrderRepository extends JpaRepository<Order, Long> {

}
```

Save with **Ctrl + S**.

Your project should now look like this:

```
user-service
├── com.example.userservice
│   ├── controller
│   │   └── UserController.java
│   ├── service
│   │   └── UserService.java
│   ├── entity
│   │   └── User.java
│   └── repository
```

└─ UserRepository.java

order-service

└─ com.example.orderservice

├─ controller

│ └─ OrderController.java

├─ service

│ └─ OrderService.java

├─ entity

│ └─ Order.java

└─ repository

└─ OrderRepository.java

JpaRepository does

JpaRepository<User, Long>

means Spring Data JPA automatically provides operations such as:

- Save a user
- Find a user
- Find all users
- Update a user
- Delete a user

The same applies to OrderRepository.

Step 7 — Connect Service Layer with Repository

Now we will connect the **Service** classes to the **Repository** classes so the application can actually save and retrieve data from MySQL.

We will do this for both:

- UserService
- OrderService

Part 1 — User Service

Open:

user-service

→ src/main/java

→ com.example.userservice

→ service

→ UserService.java

Replace the existing code with:

```
package com.example.userservice.service;
```

```
import java.util.List;
```

```
import org.springframework.stereotype.Service;
```

```
import com.example.userservice.entity.User;
```

```
import com.example.userservice.repository.UserRepository;
```

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```

```
    public UserService(UserRepository userRepository) {
```

```
        this.userRepository = userRepository;
```

```
    }
```

```
    public User saveUser(User user) {
```

```
        return userRepository.save(user);
```

```
    }
```

```
    public List<User> getAllUsers() {
```

```
        return userRepository.findAll();
```

```
    }
```

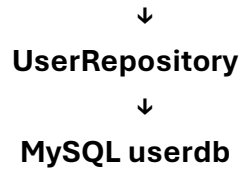
```
}
```

It Does,

UserController

↓

UserService



Part 2 — Update User Controller

Open:

user-service

→ controller

→ UserController.java

Replace the code with:

```
package com.example.userservice.controller;
```

```
import java.util.List;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.PostMapping;
```

```
import org.springframework.web.bind.annotation.RequestBody;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
import com.example.userservice.entity.User;
```

```
import com.example.userservice.service.UserService;
```


@RestController

```
public class UserController {
```

```
    private final UserService userService;
```

```
    public UserController(UserService userService) {
```

```
        this.userService = userService;
```

```
    }
```

```
    @PostMapping("/users")
```

```
    public User saveUser(@RequestBody User user) {
```

```
        return userService.saveUser(user);
```

```
    }
```

```
    @GetMapping("/users")
```

```
    public List<User> getAllUsers() {
```

```
        return userService.getAllUsers();
```

```
    }
```

```
}
```

Now your User Service can:

- **POST /users** → save a user
- **GET /users** → retrieve all users

Part 3 — Order Service

Open:

order-service

→ service

→ OrderService.java

Replace the code with:

```
package com.example.orderservice.service;
```

```
import java.util.List;
```

```
import org.springframework.stereotype.Service;
```

```
import com.example.orderservice.entity.Order;
```

```
import com.example.orderservice.repository.OrderRepository;
```

```
@Service
```

```
public class OrderService {
```

```
    private final OrderRepository orderRepository;
```

```
public OrderService(OrderRepository orderRepository) {  
    this.orderRepository = orderRepository;  
}  
  
public Order saveOrder(Order order) {  
    return orderRepository.save(order);  
}  
  
public List<Order> getAllOrders() {  
    return orderRepository.findAll();  
}  
}
```

Part 4 — Update Order Controller

Open:

order-service

→ controller

→ OrderController.java

Replace it with:

```
package com.example.orderservice.controller;
```

```
import java.util.List;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.PostMapping;
```

```
import org.springframework.web.bind.annotation.RequestBody;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
import com.example.orderservice.entity.Order;
```

```
import com.example.orderservice.service.OrderService;
```

```
@RestController
```

```
public class OrderController {
```

```
    private final OrderService orderService;
```

```
    public OrderController(OrderService orderService) {
```

```
        this.orderService = orderService;
```

```
    }
```

```
    @PostMapping("/orders")
```

```
    public Order saveOrder(@RequestBody Order order) {
```

```

        return orderService.saveOrder(order);
    }

    @GetMapping("/orders")
    public List<Order> getAllOrders() {
        return orderService.getAllOrders();
    }
}

```

Final structure

Your projects should now look like:

user-service

```

└─ com.example.userservice
    │
    │   └─ controller
    │       └─ UserController.java
    │
    │   └─ service
    │       └─ UserService.java
    │
    │   └─ entity
    │       └─ User.java
    │
    └─ repository
        └─ UserRepository.java

```

order-service

```
└─ com.example.orderservice
    │
    │   └─ controller
    │       └─ OrderController.java
    │
    │   └─ service
    │       └─ OrderService.java
    │
    │   └─ entity
    │       └─ Order.java
    │
    └─ repository
        └─ OrderRepository.java
```

Test endpoints:

User Service — port 8080

GET <http://localhost:8080/users>

POST <http://localhost:8080/users>

Order Service — port 8081

GET <http://localhost:8081/orders>

POST <http://localhost:8081/orders>

For a POST request, you can send:

{

```
"name": "Raju",  
"email": "raju@example.com"  
}
```

for User Service,

and:

```
{  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

for Order Service.

Step 8 — Test the Microservices

Now we will test both services to make sure everything created in Steps 1–7 is working correctly.

1. Start MySQL

Make sure **MySQL Server is running**.

Your databases should already exist:

userdb

orderdb

2. Start user-service

In Eclipse:

Right-click `UserServiceApplication.java` → **Run As** → **Spring Boot App**

Check the Console for:

Started UserServiceApplication

User Service should run on:

<http://localhost:8080>

3. Test User Service

Open your browser:

<http://localhost:8080/users>

Initially, you should see:

```
[]
```

Now use **Postman** to create a user.

POST

<http://localhost:8080/users>

Select:

Body → **raw** → **JSON**

Enter:

```
{  
  
  "name": "Raju",  
  
  "email": "raju@gmail.com"  
}
```


Click **Send**.

You should receive something similar to:

```
{  
  "id": 1,  
  "name": "Raju",  
  "email": "raju@gmail.com"  
}
```

Then test:

GET <http://localhost:8080/users>

You should see the saved user.

4. Start order-service

Before starting it, make sure order-service has:

```
server.port=8081
```

Then:

Right-click OrderServiceApplication.java → Run As → Spring Boot App

Check the Console for:

Started OrderServiceApplication

Order Service should run on:

<http://localhost:8081>

5. Test Order Service

Use Postman:

POST

<http://localhost:8081/orders>

Body → raw → JSON:

```
{  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

Click **Send**.

You should receive:

```
{  
  "id": 1,  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

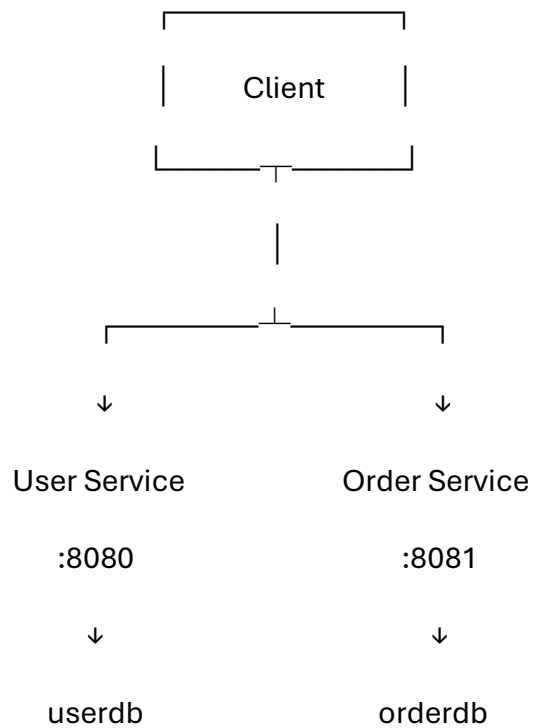
Then test:

GET <http://localhost:8081/orders>

You should see the saved order.

6. Final Step 8 check

Your complete flow is now:



Step 9 — Test and Verify the Complete Microservices Application

For the final step, verify that **User Service** and **Order Service** work correctly with their databases.

1. Check User Service

Make sure user-service is running on:

<http://localhost:8080>

Test:

GET <http://localhost:8080/users>

You should get the users stored in userdb.

2. Check Order Service

Make sure order-service is running on:

<http://localhost:8081>

Test:

GET <http://localhost:8081/orders>

You should get the orders stored in orderdb.

3. Verify the database

Open MySQL Workbench and check:

USE userdb;

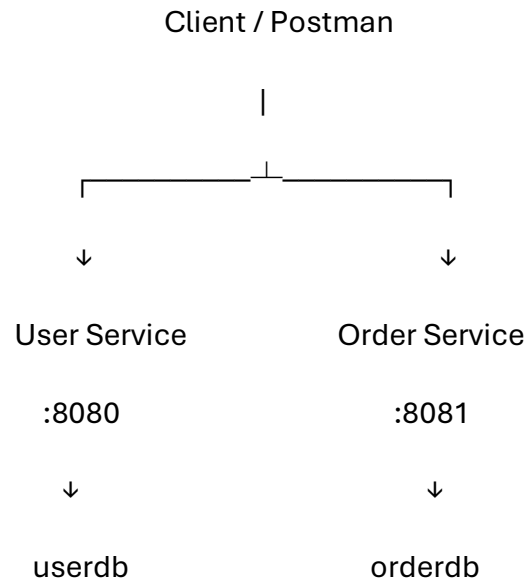
SELECT * FROM user;

And,

USE orderdb;

SELECT * FROM `order`;

4. Verify the complete flow



Step 9 checklist

- ☒ User Service starts successfully
- ☒ Order Service starts successfully
- ☒ POST /users works
- ☒ GET /users works
- ☒ POST /orders works
- ☒ GET /orders works
- ☒ Data is saved in MySQL
- ☒ Both microservices run on separate ports

Step 9 is complete.

Step 10 — Finalize and Verify the Microservices Project

For **Step 10**, complete the project by checking that everything is working together.

1. Check both projects in Eclipse

You should have:

Microservices Project

```
|
|
| └─ user-service
|
|   └─ controller
|
|     └─ service
|
|       └─ entity
|
|         └─ repository
|
|
└─ order-service
    └─ controller
    └─ service
    └─ entity
    └─ repository
```

2. Run both applications

Start:

UserServiceApplication

↓

localhost:8080

And,

OrderServiceApplication

↓

localhost:8081

Make sure there are **no errors in either Eclipse Console**.

3. Final verification

Test:

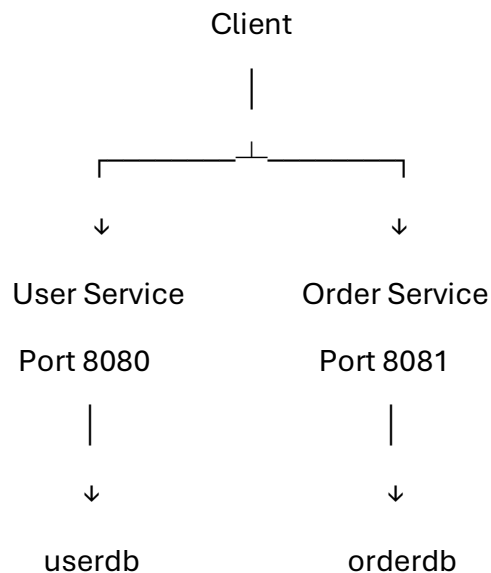
GET <http://localhost:8080/users>

And

GET <http://localhost:8081/orders>

If both return data successfully, your microservices are working.

4. Final architecture



The User Service and Order Service are created, connected to their databases, and tested.

Step 11 — Add Exception Handling

We will add exception handling to both **User Service** and **Order Service**.

Part 1 — User Service

In Eclipse, open:

user-service

→ src/main/java

→ com.example.userservice

Create a new package:

com.example.userservice.exception

Right-click the package → **New** → **Class**

Name it:

UserNotFoundException

Add,

```
package com.example.userservice.exception;
```

```
public class UserNotFoundException extends RuntimeException {
```

```
    public UserNotFoundException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

Part 2 — Create Global Exception Handler

In the same exception package:

Right-click → New → Class

Name:

GlobalExceptionHandler

Add,

```
package com.example.userservice.exception;
```

```
import org.springframework.http.HttpStatus;
```

```
import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
import org.springframework.web.bind.annotation.ResponseStatus;
```

```
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(UserNotFoundException.class)
```

```
    @ResponseStatus(HttpStatus.NOT_FOUND)
```

```
    public String handleUserNotFound(UserNotFoundException ex) {
```

```
        return ex.getMessage();
```

```
    }
```

```
    @ExceptionHandler(Exception.class)
```

```
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
```

```
public String handleGeneralException(Exception ex) {  
    return "Something went wrong";  
}  
}
```

Part 3 — Order Service

Open:

order-service

→ src/main/java

→ com.example.orderservice

Create package:

com.example.orderservice.exception

Create class:

OrderNotFoundException

Add

```
package com.example.orderservice.exception;
```

```
public class OrderNotFoundException extends RuntimeException {
```

```
    public OrderNotFoundException(String message) {  
        super(message);  
    }  
}
```

Now create another class:

GlobalExceptionHandler

Add,

```
package com.example.orderservice.exception;
```

```
import org.springframework.http.HttpStatus;
```

```
import org.springframework.web.bind.annotation.ExceptionHandler;
```

```
import org.springframework.web.bind.annotation.ResponseStatus;
```

```
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
@RestControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(OrderNotFoundException.class)
```

```
    @ResponseStatus(HttpStatus.NOT_FOUND)
```

```
    public String handleOrderNotFound(OrderNotFoundException ex) {
```

```
        return ex.getMessage();
```

```
    }
```

```
    @ExceptionHandler(Exception.class)
```

```
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
```

```
    public String handleGeneralException(Exception ex) {
```

```
        return "Something went wrong";  
    }  
}
```

Final structure:

user-service

```
└─ com.example.userservice  
    └─ exception  
        ├── UserNotFoundException.java  
        └─ GlobalExceptionHandler.java
```

order-service

```
└─ com.example.orderservice  
    └─ exception
```

└─ **OrderNotFoundException.java**

└─ **GlobalExceptionHandler.java**

Test

Restart both Spring Boot applications.

If an exception occurs, instead of showing a long error page, the API will return a controlled response such as:

User not found

Or:

Something went wrong

Step 12 — Test All APIs Using Postman

We will test both microservices one by one.

1. Start both services

In Eclipse, run:

User Service

UserServiceApplication

Port: 8080

Order Service

OrderServiceApplication

Port: 8081

Make sure both show **Started successfully** in the Eclipse Console.

2. Test User Service

Open **Postman**.

API 1 — Create User

Select:

POST

<http://localhost:8080/users>

Go to:

Body → raw → JSON

Enter:

```
{  
  
  "name": "Raju",  
  
  "email": "raju@gmail.com"  
}
```

Click **Send**.

Expected response:

```
{  
  
  "id": 1,  
  
  "name": "Raju",
```

```
"email": "raju@gmail.com"
}
```

API 2 — Get Users

Select:

GET

<http://localhost:8080/users>

Click **Send**.

Expected:

```
[
  {
    "id": 1,
    "name": "Raju",
    "email": "raju@gmail.com"
  }
]
```

3. Test Order Service

API 3 — Create Order

Select:

POST

<http://localhost:8081/orders>

Go to:

Body → raw → JSON

Enter:

```
{  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

Click **Send**.

Expected:

```
{  
  "id": 1,  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

API 4 — Get Orders

Select:

GET

<http://localhost:8081/orders>

Click **Send**.

Expected:

```
[  
  
  {  
  
    "id": 1,  
  
    "userId": 1,  
  
    "product": "Laptop",  
  
    "amount": 50000  
  
  }  
]
```

4. Test Exception Handling

Try an invalid request, for example:

GET

<http://localhost:8080/users/999>

If you haven't created that endpoint yet, don't worry about this particular test. Your existing GET /users and POST /users endpoints are the APIs currently implemented.

Postman collection

You should have these requests:

 Microservices

User Service

└─ POST /users

└─ GET /users

Order Service

└─ POST /orders

└─ GET /orders

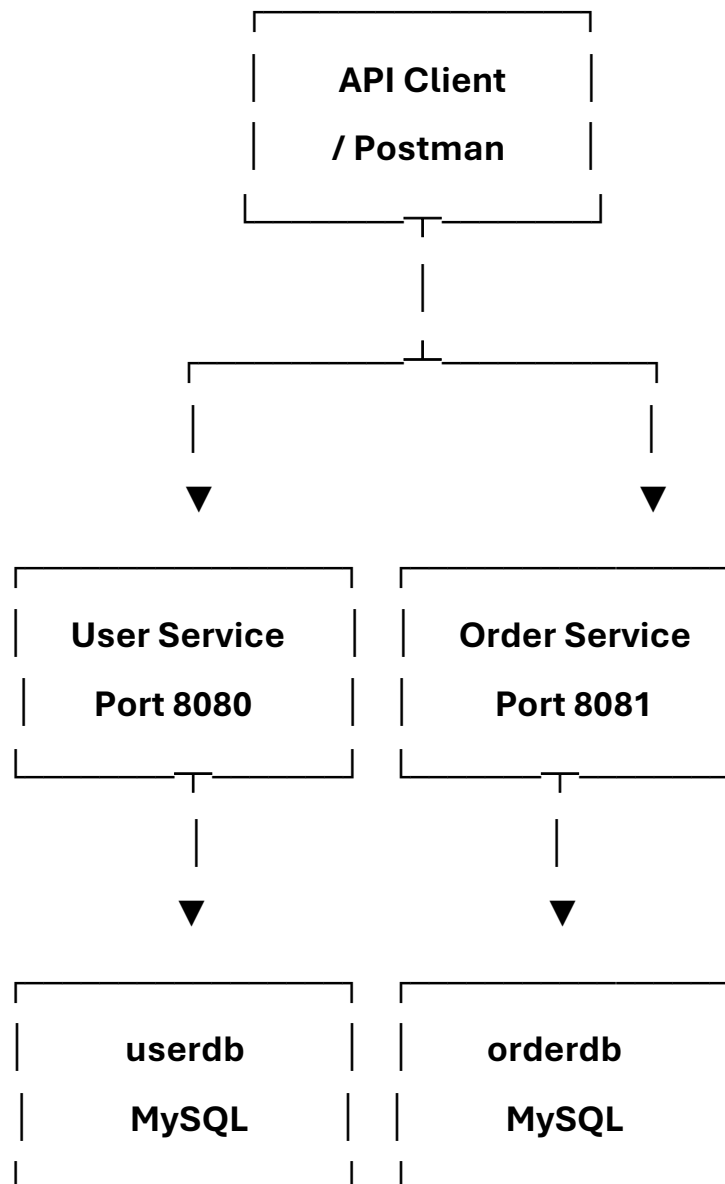
Final checklist

API	Method	Expected
/users	POST	User created
/users	GET	Users displayed
/orders	POST	Order created
/orders	GET	Orders displayed

Testing all APIs using Postman is completed.

Step 13 — Create Architecture Diagram

For our project, we have to create a diagram showing the **API Client, User Service, Order Service, and databases**.



You can use draw.io / diagrams.net, PowerPoint, or another diagram tool.

Create these 6 boxes:

1. API Client / Postman
2. User Service
3. Order Service
4. userdb
5. orderdb
6. MySQL Database if you want to show the database platform separately.

Connect them with arrows:

API Client → User Service → userdb

API Client → Order Service → orderdb

If your assignment specifically wants to show **inter-service communication**, also add:

User Service —REST—→ Order Service

Final diagram should communicate

Client

|

|→ **User Service** → **User Database**

|

|→ **Order Service** → **Order Database**

↑

|

REST communication

Save the diagram as: architecture-diagram.pdf

Step 14 — Write README

You need to create a **README.md** file in your microservices project and document what you built.

1. Create README.md

In Eclipse, right-click your main project folder/workspace folder → **New** → **File**.

Name it:

README.md

2. Add this content

Copy and paste the following into README.md:

Java Spring Boot Microservices System

Project Overview

This project demonstrates a basic microservices architecture using Java and Spring Boot.

The application contains two independent services:

- User Service
- Order Service

Architecture

API Client / Postman |

| | v v User Service Order Service | | v v userdb orderdb (MySQL) (MySQL)

User Service also communicates with Order Service using REST APIs.

Technologies Used

- Java 17
- Spring Boot
- Spring Web
- Spring Data JPA
- MySQL
- Maven
- Postman
- Eclipse IDE

Services

User Service

Port:

8080

Endpoints:

POST /users

GET /users

Database:

userdb

Order Service

Port:

8081

Endpoints:

POST /orders

GET /orders

Database:

orderdb

User Service Example

Create a user:

POST <http://localhost:8080/users>

Request:

```
{  
  "name": "Raju",  
  "email": "raju@gmail.com"  
}
```

Order Service Example

Create an order:

POST <http://localhost:8081/orders>

Request:

```
{  
  "userId": 1,  
  "product": "Laptop",  
  "amount": 50000  
}
```

Database Configuration

The User Service uses the userdb MySQL database.

The Order Service uses the orderdb MySQL database.

Testing

All APIs can be tested using Postman.

The project includes:

- User creation and retrieval
- Order creation and retrieval
- Database persistence
- Exception handling
- REST-based communication

How to Run

1. Start MySQL.
2. Create userdb and orderdb.
3. Open both projects in Eclipse.
4. Start UserServiceApplication.
5. Start OrderServiceApplication.
6. Test the APIs using Postman.

Project Structure

```
user-service
├── controller
├── service
├── entity
├── repository
└── exception
```

```
order-service
├── controller
├── service
└── entity
```



```
| repository
| exception
```

Author

Raju

3. Save the file

Press:

Ctrl + S

Your README should now be on the project.

4. Check the final project

You should have:

Microservices Project

```
|
|
|— user-service
|— order-service
|— README.md
|— architecture-diagram.png
```

Step 15: “Push to Git”

